INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES
DE MONTERREY
CAMPUS GUADALAJARA

Curso de ROS 2
Apuntes de clase

Impartido por:

Dr. Eduardo de Jesús Dávila Meza

para el bloque integrador de:

Robótica y Sistemas Inteligentes

para estudiantes de:

Ingeniería en Robótica y Sistemas Digitales

de la Escuela de:

Ingeniería y Ciencias

Zapopan, Jalisco, México. Marzo - Junio 2026.

Contents

Contents iii

List of Acronyms and Special Terms vii

1 ROS 2 Environment Setup and Development Tools 1
- 1.1 Introduction to ROS 2 . . . 1
  - 1.1.1 Basic Concepts . . . 1
  - 1.1.2 Some ROS Distributions . . . 2
  - 1.1.3 ROS Distribution for the Course . . . 2
- 1.2 Installation of Ubuntu 22 and ROS 2 Humble Hawksbill . . . 3
  - 1.2.1 Installing Ubuntu 22.04.5 LTS (Jammy Jellyfish) . . . 3
  - 1.2.2 First Steps After Installing Ubuntu 22 . . . 3
  - 1.2.3 Installing ROS 2 Humble Hawksbill . . . 4
- 1.3 Try Some Examples . . . 5
  - 1.3.1 Talker-Listener . . . 5
  - 1.3.2 Turtlesim: Publish and Move a Turtle . . . 6
- 1.4 `colcon` Configuration . . . 6
  - 1.4.1 Installing `colcon` . . . 6
  - 1.4.2 Enabling `colcon` Argument Completion . . . 6
- 1.5 Configuring the Development Environment . . . 7
  - 1.5.1 Installing Visual Studio Code . . . 7
  - 1.5.2 Installing Recommended VS Code Extensions . . . 7
  - 1.5.3 Installing Terminator for Multiple Terminals . . . 8
- 1.6 Practice Assignment: Running ROS 2 Examples . . . 10
  - 1.6.1 Examples to Try . . . 10
  - 1.6.2 Submission Instructions . . . 10

2 ROS 2 Workspace, Package, and Node Development 13
- 2.1 Fundamental Concepts of ROS 2 . . . 13
  - 2.1.1 Workspace . . . 13
  - 2.1.2 Packages . . . 13
  - 2.1.3 Nodes . . . 14
  - 2.1.4 Topics . . . 14
- 2.2 Workspace Creation and Setup . . . 15
  - 2.2.1 Creating the Workspace . . . 15
  - 2.2.2 Building the Workspace . . . 16
  - 2.2.3 Sourcing the Workspace Environment . . . 16

2.3 Creating a ROS 2 Package for C++ Nodes . . . 16
2.3.1 Navigating to the src Directory . . . 16
2.3.2 Creating the C++ Package . . . 17
2.3.3 Building the Package . . . 17
2.3.4 Examining the Package Contents . . . 17
2.3.5 Detailed Examination of Key Package Files . . . 17
2.3.6 Ensuring Consistency Between package.xml and CMakeLists.txt . . . 18
2.4 Creating a ROS 2 Package for Python Nodes . . . 18
2.4.1 Navigating to the src Directory . . . 18
2.4.2 Creating the Python Package . . . 18
2.4.3 Building the Package . . . 19
2.4.4 Examining the Package Contents . . . 19
2.4.5 Detailed Examination of Key Package Files . . . 19
2.4.6 Ensuring Consistency Between package.xml and setup.py . . . 20
2.5 Creating a ROS 2 Publisher Node with C++ . . . 20
2.5.1 Setting Up the C++ Node . . . 20
2.5.2 Editing the C++ Node . . . 21
2.5.3 Integrating the Publisher Node into the Package . . . 24
2.5.4 Running the Publisher Node as a Standalone Executable (Optional) . . . 25
2.5.5 Running the ROS 2 Publisher Node with ROS 2 . . . 26
2.5.6 Summary of Key Identifiers . . . 26
2.6 Creating a ROS 2 Subscriber Node with C++ . . . 26
2.6.1 Setting Up the C++ Node . . . 26
2.6.2 Editing the C++ Node . . . 27
2.6.3 Integrating the Subscriber Node into the Package . . . 29
2.6.4 Running the Subscriber Node as a Standalone Executable (Optional) . . . 30
2.6.5 Running the ROS 2 Subscriber Node with ROS 2 . . . 31
2.6.6 Summary of Key Identifiers . . . 31
2.7 Creating a ROS 2 Publisher Node with Python . . . 31
2.7.1 Setting Up the Python Node . . . 31
2.7.2 Editing the Python Node . . . 32
2.7.3 Running the Publisher Node Standalone (Optional) . . . 35
2.7.4 Integrating the Publisher Node into the Package . . . 35
2.7.5 Running the ROS 2 Publisher Node with ROS 2 . . . 36
2.7.6 Summary of Key Identifiers . . . 36
2.8 Creating a ROS 2 Subscriber Node with Python . . . 37
2.8.1 Setting Up the Python Node . . . 37
2.8.2 Editing the Python Node . . . 37
2.8.3 Breaking Down the Subscriber Node Code . . . 38
2.8.4 Running the Subscriber Node Standalone (Optional) . . . 40
2.8.5 Integrating the Subscriber Node into the Package . . . 40
2.8.6 Running the ROS 2 Subscriber Node with ROS 2 . . . 41
2.8.7 Summary of Key Identifiers . . . 41
2.9 Practice Assignment: Running ROS 2 Nodes . . . 42
2.9.1 Executing ROS 2 Publishers and Subscribers . . . 42

2.9.2 Submission Instructions . . . 43

3 ROS 2 Service and Client 45

A Essential ROS 2 Command Reference 47
A.1 System Package Management and Updates in Ubuntu . . . 47
A.2 ROS 2 Environment Setup . . . 48
A.3 Workspace Creation and Building . . . 48
A.4 Package Creation and Editing in C++ and Python . . . 49
A.5 Dev & Debug Essentials: CLI/GUI Commands . . . 50

B Using the `geometry_msgs` Package in C++ and Python 51
B.1 Importing the Package . . . 51
B.1.1 In C++ . . . 51
B.1.2 In Python . . . 51
B.1.3 Including `geometry_msgs` in Your Package Dependencies . . . 52
B.2 Using Message Definitions . . . 52
B.2.1 Point . . . 52
B.2.2 Quaternion . . . 53
B.2.3 Pose . . . 53
B.3 Additional Message Types . . . 54

C Configuring ROS 2 Package Paths for URDF Visualizer in VS Code 55
C.1 Steps to Add a ROS 2 Package Path . . . 55
C.2 Additional Notes . . . 56

List of Acronyms and Special Terms

Acronyms

S

SLAM simultaneous localization and mapping. — Pages: 173, 174, 181, 182, 186.

CHAPTER 1

ROS 2 Environment Setup and Development Tools

Dr. Eduardo de Jesús Dávila Meza
EduardoDavilaMeza
eduardo.davila.meza@tec.mx
eDavila-DrRaccoon

1.1 Introduction to ROS 2

1.1.1 Basic Concepts

The Robot Operating System (ROS) is not an actual operating system but a flexible *framework* for writing robot software. It provides a collection of software libraries, tools, and conventions to simplify the task of creating complex and robust robot applications. From drivers and state-of-the-art algorithms to powerful developer tools, ROS has the open source tools you need for your next robotics project. Some of its main advantages include:

- **Multi-language:** While ROS supports multiple programming languages, the primary supported languages are C++ and Python. Support for other languages like Java exists but is less common.
- **Free and open-source:** ROS 1 and ROS 2 are available on Ubuntu, with ROS 2 also supporting Windows and macOS. However, the availability and stability can vary across different versions and platforms.
- **Support:** Since ROS was started in 2007, a lot has changed in the robotics and ROS community. The ROS 2 project began in 2015 to address the evolving needs of the robotics community, building upon the strengths of ROS 1 and introducing improvements for better performance, security, and support for real-time systems.

1.1.2 Some ROS Distributions

ROS is released as distributions (or "distros"), with several versions supported concurrently. Some releases come with long-term support (LTS), meaning they are more stable and have undergone extensive testing, while other distributions are newer, have shorter lifetimes, and support more recent platforms and package versions. Generally, a new ROS distro is released every year on World Turtle Day, with LTS releases appearing in even-numbered years. ROS is available for different versions of Ubuntu and other operating systems. Some of the notable distributions include:

- **Indigo Igloo (ROS 1):** Ubuntu 14.04 (Trusty Tahr)
- **Kinetic Kame (ROS 1):** Ubuntu 16.04 (Xenial Xerus)
- **Melodic Morenia (ROS 1):** Ubuntu 18.04 (Bionic Beaver)
- **Noetic Ninjemys (ROS 1):** Ubuntu 20.04 (Focal Fossa)
- **Foxy Fitzroy (ROS 2):** Ubuntu 20.04 (Focal Fossa)
- **Humble Hawksbill (ROS 2):** Ubuntu 22.04 (Jammy Jellyfish)
- **Jazzy Jalisco (ROS 2):** Ubuntu 24.04 (Noble Numbat)

For more details, see the ROS Distributions list. Currently, the Noetic Ninjemys, Humble Hawksbill, and Jazzy Jalisco distributions are actively supported.

1.1.3 ROS Distribution for the Course

For this course, we will use **ROS 2 Humble Hawksbill**, a long-term support (LTS) release that offers enhanced performance, security, and real-time capabilities. Its logo, shown in Figure 1.1, reflects its distinctive branding. ROS 2 Humble Hawksbill is compatible with Ubuntu 22.04 (Jammy Jellyfish) and Windows 10, making it a versatile choice for both simulation and deployment on physical hardware. This distribution will serve as the foundation for all class exercises and, in particular, for project development.

Figure 1.1: ROS 2 Humble Hawksbill Logo.

1.2 Installation of Ubuntu 22 and ROS 2 Humble Hawksbill

1.2.1 Installing Ubuntu 22.04.5 LTS (Jammy Jellyfish)

1. **Download the Universal USB Installer**
   Visit the Uptodown website to download the Universal USB Installer, version `2.0.1.4`, on Windows.
2. **Download the Ubuntu ISO**
   Download the Ubuntu 22.04.5 LTS (Jammy Jellyfish) ISO - *64-bit PC (AMD64) desktop image*, from the official release page: Ubuntu releases.
3. **Create a Bootable USB Drive**
   Insert a USB drive (minimum 8 GB) and use the Universal USB Installer tool with the downloaded ISO to create a bootable USB stick. Follow the on-screen prompts to properly set up the drive. You may also follow the instructions provided by your instructor, the official guide: Universal USB Installer, or refer to this tutorial: YouTube Guide.
4. **Prepare Your Machine (if dual-booting with Windows)**
   If you are keeping Windows and installing Ubuntu alongside it, you should free some unallocated disk space before installation:
   - Defragment the `C:` drive.
   - Additionally, you can shrink the Windows partition to create at least 32 GB of unallocated space (advanced installation).
   - If the volume reduction fails, delete old restore points to free up additional space. See, for example, this tutorial: Windows - Shrinking hard disk volume to create new partition.
5. **Install Ubuntu on Your Machine**
   Boot the target computer from the USB drive. Follow the Ubuntu installation wizard to install Ubuntu 22.04 LTS and configure your system settings as needed:
   - Select language and keyboard layout.
   - Connect to a network (optional during installation).
   - Choose installation type (*Normal* or *Minimal*).
   - Allocate disk space (at least 64 GB) if you want to *Install Ubuntu alongside Windows* or select *Erase disk and install Ubuntu*.
   - Create a user account, password, and set the hostname.

1.2.2 First Steps After Installing Ubuntu 22

Open a terminal from the application menu by typing `terminal` or by pressing `Ctrl+Alt+T`, and then:

- Update the system:

```
$ sudo apt update && sudo apt upgrade -y
```

- Check the installed Python version:

```
$ python3 --version
```

1.2.3 Installing ROS 2 Humble Hawksbill

Recommended Method: Using Debian Packages

The official ROS 2 Humble Hawksbill documentation recommends installing from deb packages for a stable and straightforward setup. Follow the installation instructions on the official ROS 2 Humble Hawksbill Installation Guide. This method installs pre-built binaries and generally covers all core dependencies needed for running ROS 2.

Instructions:

Set locale
Make sure you have a locale which supports UTF-8. The following settings are tested, although any UTF-8 supported locale should work.

```
$ locale  # check for UTF-8

$ sudo apt update && sudo apt install locales
$ sudo locale-gen en_US en_US.UTF-8
$ sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
$ export LANG=en_US.UTF-8

$ locale  # verify settings
```

Setup Sources
You will need to add the ROS 2 apt repository to your system. First, ensure that the Ubuntu Universe repository is enabled.

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
```

Now, add the ROS 2 GPG key with apt:

```
$ sudo apt update && sudo apt install curl -y
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
↪  /usr/share/keyrings/ros-archive-keyring.gpg
```

Then, add the repository to your sources list:

```
$ echo "deb [arch=$(dpkg --print-architecture)
↪  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
↪  http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME)
↪  main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Install ROS 2 Packages
Update your apt repository caches after setting up the repositories:

```
$ sudo apt update
```

ROS 2 packages are built on frequently updated Ubuntu systems. It is always recommended to ensure your system is up to date before installing new packages:

```
$ sudo apt upgrade
```

⚠ Warning:

Due to early updates in Ubuntu 22.04, it is important that `systemd` and `udev`-related packages are updated before installing ROS 2. Installing ROS 2's dependencies on a freshly installed system without upgrading can trigger the removal of critical system packages. Please refer to ros2/ros2#1272 and Launchpad #1974196 for more information.

Desktop Install (Recommended): This includes ROS, RViz, demos, and tutorials.

```
$ sudo apt install ros-humble-desktop
```

Development Tools: Compilers and other tools to build ROS packages.

```
$ sudo apt install ros-dev-tools
```

Environment Setup – Sourcing the Setup Script

Set up your environment by sourcing the following file (replace `.bash` with your shell if not using bash):

```
$ source /opt/ros/humble/setup.bash
```

Alternative: Building from Source

If you require customizations or intend to develop complex packages, you can also install ROS 2 Humble Hawksbill from source. Note that building from source may require additional dependency installations (similar to ROS1). For most beginners and for the purposes of this course, the deb package installation is recommended.

Sourcing the Setup Script

After installing ROS 2, source the setup script to ensure your environment is correctly configured:

```
$ source /opt/ros/humble/setup.bash
```

Optionally, add the sourcing command to your shell's initialization file (e.g., `.bashrc`) so that the ROS environment variables are automatically loaded every time a new terminal session is opened (replace `.bash` with your shell if not using bash).

```
$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
$ source ~/.bashrc
```

1.3 Try Some Examples

1.3.1 Talker-Listener

If you installed `ros-humble-desktop`, try some examples. In one terminal, source the setup file and run a C++ talker:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_cpp talker
```

In another terminal, source the setup file and run a Python listener:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_py listener
```

You should see the talker publishing messages and the listener confirming reception, verifying that both the C++ and Python APIs are working properly.

1.3.2 Turtlesim: Publish and Move a Turtle

Another example you can try is the `turtlesim` package, which demonstrates basic ROS 2 communication using a simulated turtle.

First, ensure that the package is installed:

```
$ sudo apt install ros-humble-turtlesim
```

Then, in a new terminal, launch the turtlesim node:

```
$ ros2 run turtlesim turtlesim_node
```

Next, open another terminal, use the `teleop` node to control the turtle with your keyboard:

```
$ ros2 run turtlesim turtle_teleop_key
```

Now, pressing the arrow keys will move the turtle in the corresponding direction. This example demonstrates publishing velocity commands to a topic that the turtlesim node subscribes to, allowing interaction with the simulated environment.

1.4 `colcon` Configuration

`colcon` is the command-line tool used in ROS 2 to build sets of packages in a workspace. It automatically detects packages (via `package.xml`), resolves dependencies, and builds them (often in parallel) to simplify the development process. `colcon` also supports options like `--symlink-install` for faster iterative development.

1.4.1 Installing `colcon`

Update your package list and ensure that the common `colcon` extensions are installed. Although these packages are usually installed as part of the `ros-humble-desktop` and `ros-dev-tools` installations, it is good practice to verify their presence by running the following commands:

```
$ sudo apt update
$ sudo apt install python3-colcon-common-extensions
```

1.4.2 Enabling `colcon` Argument Completion

To simplify command usage with `colcon`, add the argument completion script to your shell initialization file (e.g., `.bashrc`). Execute the following commands:

```
$ echo "source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash" >> ~/.bashrc
$ source ~/.bashrc
```

You can verify that the sourcing command was added by viewing your `.bashrc` file:

```
$ cat ~/.bashrc
```

1.5 Configuring the Development Environment

With the following tools and extensions, your development environment will be well-equipped for Python projects.

1.5.1 Installing Visual Studio Code

Visual Studio Code (VS Code) is a versatile code editor that supports various programming languages and development environments. To install it on Ubuntu 22.04, follow these steps:

1. **Using the Ubuntu Software Center**
   (a) Open the *Ubuntu Software* application from the application menu.
   (b) In the search bar, type `Visual Studio Code`.
   (c) Locate Visual Studio Code in the search results and click *Install*.
2. **Using the Official .deb Package**
   (a) Visit the official VS Code download page: code.visualstudio.com.
   (b) Click on the `.deb` package suitable for Debian/Ubuntu.
   (c) Once downloaded, open a terminal (`Ctrl+Alt+T`) and navigate to the directory containing the downloaded file.
   (d) Install the package using:

   ```
   $ sudo apt install ./<file>.deb
   ```

   Replace `<file>` with the actual filename.

These methods ensure that VS Code is added to your system repositories and receives updates automatically.

1.5.2 Installing Recommended VS Code Extensions

Enhance your development experience by installing the following VS Code extensions:

- **C++** by *Microsoft* – Offers C++ IntelliSense, debugging, and code browsing.
- **CMake** by *twxs* – Simplifies working with CMake projects.
- **CMake Tools** by *Microsoft* – Provides CMake project integration.
- **Error Lens** by *Alexander* – Displays inline error messages in the code editor.
- **Indent-Rainbow** by *oderwat* – Highlights indentation levels with different colors.
- **Python** by *Microsoft* – Provides rich support for Python, including features such as IntelliSense, linting, and debugging.
- **Robotics Developer Environment** by *Ranch Hand Robotics LLC* – Helps build robotics solutions targeting the ROS 2.
- **URDF** by *smilerobotics* – Adds syntax highlighting and support for URDF files.

- **URDF Visualizer** by *morningfrog* – Allows to preview URDF files within the editor.
- **vscode-icons** by *VSCode Icons Team* – Adds file icons for better visual identification.
- **XML** by *Red Hat* – Enhances XML editing capabilities.
- **XML Tools** by *Josh Johnson* – Offers additional functionalities for XML files.

To install these extensions:

1. Open VS Code from the application menu by typing `code`.
2. Navigate to the *Extensions* view by clicking on the square icon () in the sidebar or pressing `Ctrl+Shift+X`.
3. Search for each extension by name and click *Install*.

1.5.3 Installing Terminator for Multiple Terminals

Terminator is a terminal emulator that allows splitting the window into multiple terminals, facilitating simultaneous operations. To install Terminator:

1. Open a terminal.
2. Update the package list:
   ```
   $ sudo apt update
   ```
3. Install Terminator:
   ```
   $ sudo apt install terminator
   ```
4. Launch Terminator from the applications menu, by typing `terminator` in the terminal, or by pressing `Ctrl+Alt+T`.

Installing Terminator may have set it as the default, so when you press `Ctrl+Alt+T` it opens Terminator instead of GNOME Terminal (or your original terminal). To achieve your goal, you can change the default and then assign Terminator to a different shortcut, as described in the next.

Reset the Default Terminal Emulator

Ubuntu uses the alternative system for the "x-terminal-emulator." You can reconfigure it so that pressing `Ctrl+Alt+T` launches your original terminal (for example, GNOME Terminal):

1. **Open a terminal** (if Terminator is the only one available, you can temporarily launch it).
2. Run the command:
   ```
   $ sudo update-alternatives --config x-terminal-emulator
   ```
3. You will see a list of installed terminal emulators. Type the number corresponding to your original terminal (e.g., `/usr/bin/gnome-terminal.wrapper`) and press **Enter**.

This sets the default terminal emulator that the system shortcut uses.

Create a Custom Shortcut for Terminator

Next, if you want a separate shortcut (like `Ctrl+Alt+E`) that launches Terminator:

1. **Open Settings** in your Ubuntu desktop.
2. Navigate to **Keyboard** (or **Keyboard Shortcuts**).
3. Scroll down to or click on **Custom Shortcuts**.
4. Click the **Add Shortcut** button.
5. Set a name (e.g., "Terminator") and for the command enter:
   ```
   terminator
   ```
6. Assign the new shortcut to `Ctrl+Alt+E` (click the shortcut key field and press the keys).
7. Save your new shortcut.

Now:

- `Ctrl+Alt+T` will launch your default terminal (GNOME Terminal),
- `Ctrl+Alt+E` will launch Terminator.

This way, you keep your usual terminal for everyday tasks and have Terminator available on a separate key combination for when you need its advanced features.

1.6 Practice Assignment: Running ROS 2 Examples

In this assignment, you will run the ROS 2 examples presented in Section 1.3 on your computer and capture evidence of successful execution using a screenshot. Submit your evidence in the designated assignment area on Canvas (within the ROS 2 module) as a PNG file. The filename must follow this format:

`FirstnameLastname_evidence_sX.png`,

where `sX` indicates the session number, for example:

`EduardoDavila_evidence_s1.png`.

1.6.1 Examples to Try

Talker-Listener

After installing `ros-humble-desktop` (and optionally `Terminator`), try the following:

- In **Terminal 1**, source the setup file and run a C++ talker:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_cpp talker
```

- In **Terminal 2**, source the setup file and run a Python listener:

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_py listener
```

You should observe that the talker publishes messages and the listener confirms their reception.

Turtlesim: Publish and Move a Turtle

Another example is provided by the `turtlesim` package:

- In **Terminal 3**, run the turtlesim node:

```
$ ros2 run turtlesim turtlesim_node
```

- In **Terminal 4**, run the teleoperation node to control the turtle:

```
$ ros2 run turtlesim turtle_teleop_key
```

Use the arrow keys to move the turtle. This example demonstrates publishing velocity commands to control the simulated turtle.

1.6.2 Submission Instructions

1. Run the examples as described above.
2. Capture a **screenshot showing the terminal output** where the examples are running successfully (see Figure 1.2 for an example).
3. Save the screenshot in PNG format.

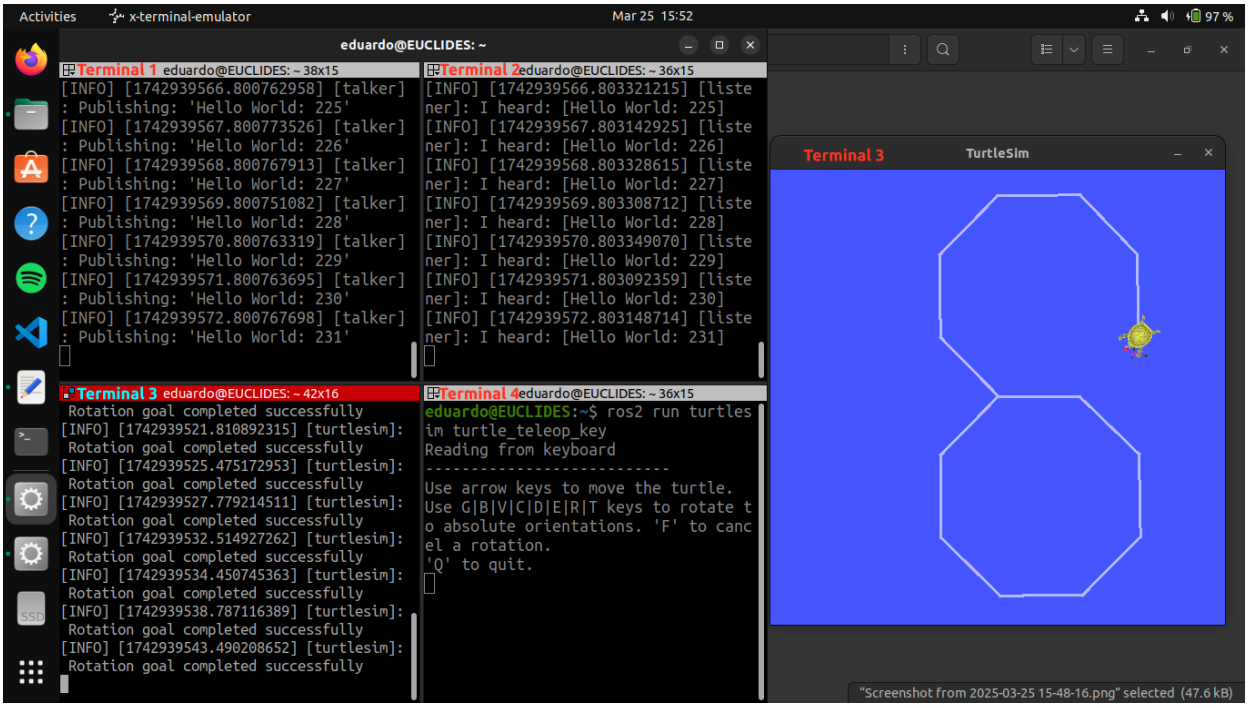

Figure 1.2: Example screenshot showing terminal outputs from both the Talker-Listener example and the Turtlesim node example in action.

4. Name the file according to the format: `FirstnameLastname_evidence_sX.png` (replace `X` with the session number).
5. Submit your screenshot file as instructed by the course guidelines.

Note: Your machine's username and device name must be visible in the screenshot to verify the authenticity of the submission, as shown in Figure 1.2. Evidence that appears copied, unclear, or altered will be considered invalid and may result in a score of 0 for this assignment.

CHAPTER 2

ROS 2 Workspace, Package, and Node Development: Publisher and Subscriber

Dr. Eduardo de Jesús Dávila Meza
EduardoDavilaMeza
eduardo.davila.meza@tec.mx
eDavila-DrRaccoon

2.1 Fundamental Concepts of ROS 2

ROS 2 is a middleware based on a strongly-typed, anonymous publish/subscribe mechanism that enables message passing between different processes. At the core of any ROS 2 system is the ROS graph, which represents the network of nodes and the connections through which they communicate.

This section introduces the fundamental concepts necessary to understand the basics of ROS 2.

2.1.1 Workspace

A ROS 2 workspace is a directory that contains one or more ROS 2 packages. It serves as the working environment for building, developing, and testing ROS 2 applications.

Best practices recommend creating a separate directory for each workspace, with a meaningful name that reflects its purpose. Inside the workspace, it is common to include a `src` folder to store ROS 2 packages, ensuring an organized structure.

2.1.2 Packages

A ROS 2 package is the fundamental unit of software organization in ROS 2. It provides a structured way to manage code, facilitating its distribution, installation, and reuse. A package may contain:

- Nodes

- Libraries
- Configuration files
- Launch files
- Other resources necessary for execution

A single workspace can contain multiple packages, even if they use different build types (e.g., CMake, Python). However, packages cannot be nested within each other.

ROS 2 uses `ament` as its build system and `colcon` as its build tool. Packages can be created using either CMake or Python, each with minimum required contents, as detailed in Sections 2.3 and 2.4, respectively.

2.1.3 Nodes

A ROS 2 node is a fundamental component that represents a single process performing computation. Nodes communicate with each other through:

- Topics for continuous data exchange.
- Services for request-response interactions.
- Actions for long-running tasks with feedback.

Each node is part of the ROS graph and can communicate with other nodes within the same process, across different processes, or even across multiple machines. Nodes are designed to be modular, meaning that each node should focus on a single logical task.

A node can simultaneously act as:

- A publisher for sending messages.
- A subscriber for receiving messages.
- A service client for requesting a computation.
- A service server for providing a computation.
- An action clients for initiating long-running tasks with feedback.
- An action servers for executing long-running tasks with periodic feedback.

Connections between nodes are established dynamically, allowing for a modular and scalable system design.

2.1.4 Topics

Topics are a core mechanism in ROS 2 for message-based communication between nodes. They are used for continuous data exchange, such as transmitting sensor readings, robot state updates, or other real-time information.

ROS 2 follows a publish/subscribe model for topics, meaning:

- Publishers send data to a topic.
- Subscribers receive data from a topic.
- Multiple publishers and subscribers can exist on the same topic.
- Communication is anonymous, so subscribers can receive messages without knowing which publisher transmitted the data.

Publish/Subscribe Model

The publish/subscribe system allows for flexible and decoupled communication:

- Publishers and subscribers communicate via a shared topic name.
- Multiple publishers and subscribers can exist on a topic simultaneously.
- When a publisher sends data, all subscribers of that topic receive it.

This architecture is often compared to a bus system in electrical engineering, where multiple devices can share a common communication channel.

Anonymous Communication

ROS 2 implements anonymous communication, meaning:

- A subscriber does not need to know which publisher sent a message.
- Publishers and subscribers can be dynamically replaced without affecting the system.
- Debugging and monitoring tools (e.g., `ros2bagrecord`) can subscribe to topics without interrupting existing communication.

Strongly-Typed Messages

ROS 2 enforces strong typing in its publish/subscribe system to ensure data consistency and integrity. This guarantees:

1. Strict Data Types: Each field in a message adheres to a predefined data type.

   ```
   uint32 field1
   string field2
   ```

   In this example, `field1` must always be an unsigned 32-bit integer, and `field2` must always be a string.
2. Well-Defined Semantics: Message contents adhere to clear conventions. For example, an IMU message includes a 3-dimensional angular velocity vector, where each component is explicitly defined in radians per second. This ensures consistency and prevents misinterpretation of the transmitted data.

2.2 Workspace Creation and Setup

A ROS 2 workspace is a directory that contains one or more ROS 2 packages, as described in Section 2.1. When you build your workspace using `colcon`, it automatically creates the `build`, `install`, and `log` directories. This structure helps manage dependencies and package configurations efficiently, ensuring a clean and organized development environment.

2.2.1 Creating the Workspace

After installing ROS 2, set up your workspace by creating a directory (e.g., `~/ros2_ws`) and adding a `src` folder:

```
$ mkdir -p ros2_ws/src
```

The best practice is to create and organize your packages inside the `src` folder to keep the workspace structure clean and maintainable.

2.2.2 Building the Workspace

Navigate to the workspace directory:

```
$ cd ros2_ws
```

Build the workspace using `colcon`:

```
$ colcon build
```

For development purposes, you might consider using the `--symlink-install` option. This allows changes in source files to take effect immediately without requiring a full rebuild:

```
$ colcon build --symlink-install
```

If the build completes successfully, you will see the following confirmation message: `'colcon build' successful`, and the `build`, `install`, and `log` directories will be created within your workspace.

> **Note:**
> The `--symlink-install` option only creates symbolic links to the source files from Python packages. Therefore, changes to source files from C++ packages will still require a rebuild to reflect in the installed binaries.

2.2.3 Sourcing the Workspace Environment

To overlay your workspace on top of the system installation and load the environment variables for your newly built packages, run:

```
$ source ./install/setup.bash
```

Note: Unlike the ROS 2 and `colcon` setup scripts, this sourcing command is not permanently added to your shell's initialization file. For learning purposes, you must run it in each new terminal session where you work with your ROS 2 workspace.

For more details on `colcon` and workspace management, refer to the ROS 2 Colcon tutorial.

2.3 Creating a ROS 2 Package for C++ Nodes

2.3.1 Navigating to the `src` Directory

Begin by opening a terminal and navigating to the `src` directory within your ROS 2 workspace (e.g., `~/ros2_ws`):

```
$ cd ~/ros2_ws/src
```

2.3.2 Creating the C++ Package

Use the `ros2pkgcreate` command to create a new C++ package. Replace `s2_cpp_pubsub` with your desired package name and provide an appropriate description:

```
$ ros2 pkg create s2_cpp_pubsub --build-type ament_cmake --dependencies rclcpp std_msgs
↪ --license Apache-2.0 --description "Your package description here"
```

Let's break down the components of this command:

- `s2_cpp_pubsub`: Specifies the name of the new package.
- `--build-typeament_cmake`: Indicates that the package uses C++ and will be built with `ament_cmake`.
- `--dependenciesrclcppstd_msgs`: Lists the package dependencies, in this case, `rclcpp` (the ROS 2 C++ API) and `std_msgs` (standard message definitions).
- `--licenseApache-2.0`: Sets the license type for the package.
- `--description"Yourpackagedescriptionhere"`: Provides a brief description of the package.

2.3.3 Building the Package

After creating the package, navigate back to the root of your workspace and build the package using `colcon`:

```
$ cd ~/ros2_ws
$ colcon build --packages-select s2_cpp_pubsub
```

Alternatively, to build all packages in the workspace:

```
$ colcon build
```

2.3.4 Examining the Package Contents

Navigate to the newly created package directory, `s2_cpp_pubsub`, and list its contents:

```
$ cd src/s2_cpp_pubsub
$ ls
```

You should see the following key files and directories:

- `CMakeLists.txt`: Contains instructions for building the code within the package using CMake.
- `include/`: Directory containing the public header files for the package.
- `package.xml`: Defines metadata about the package, including its name, version, description, licenses, maintainers, authors, and dependencies.
- `src/`: Directory containing the source code files for the package.

2.3.5 Detailed Examination of Key Package Files

`package.xml`

The `package.xml` file contains essential metadata about the ROS 2 package. Key elements include:

- `<name>`: Specifies the package's name.
- `<version>`: Indicates the current version of the package.
- `<description>`: Provides a brief overview of the package's purpose.
- `<maintainer>`: Contains contact information for the package maintainer.
- `<license>`: Details the licensing information.
- `<depend>`: Lists build-time and runtime dependencies required by the package.

To examine the `package.xml` file, use the following command:

```
$ code package.xml
```

Ensure that the dependencies listed here match those specified during package creation.

`CMakeLists.txt`

The `CMakeLists.txt` file contains instructions for building the package using CMake. It specifies details such as the minimum required CMake version, project name, dependencies, include directories, source files, and targets to be built.

To inspect the `CMakeLists.txt` file, execute:

```
$ code CMakeLists.txt
```

2.3.6 Ensuring Consistency Between `package.xml` and `CMakeLists.txt`

It is crucial that the information in `package.xml` and `CMakeLists.txt` is consistent. Discrepancies between these files can lead to build or runtime issues. Ensure that details like the package name, version, description, maintainer, license, and dependencies align across both files.

By following these steps, you have created a C++-based ROS 2 package, built it, and examined its structure and configuration files. This foundational setup is essential for developing and organizing your ROS 2 C++ nodes effectively.

For more details on package creation and management, refer to the official Creating Your First ROS 2 Package tutorial.

2.4 Creating a ROS 2 Package for Python Nodes

2.4.1 Navigating to the `src` Directory

Begin by opening a terminal and navigating to the `src` directory within your ROS 2 workspace (e.g., `~/ros2_ws`):

```
$ cd ~/ros2_ws/src
```

2.4.2 Creating the Python Package

Use the `ros2pkgcreate` command to create a new Python package. Replace `s2_py_pubsub` with your desired package name and provide an appropriate description:

```
$ ros2 pkg create s2_py_pubsub --build-type ament_python --dependencies rclpy std_msgs
↪  --license Apache-2.0 --description "Your package description here"
```

Let's break down the components of this command:

- `s2_py_pubsub`: Specifies the name of the new package.
- `--build-typeament_python`: Indicates that the package uses Python and will be built with `ament_python`.
- `--dependenciesrclpystd_msgs`: Lists the package dependencies, in this case, `rclpy` (the ROS 2 Python API) and `std_msgs` (standard message definitions).
- `--licenseApache-2.0`: Sets the license type for the package.
- `--description"Yourpackagedescriptionhere"`: Provides a brief description of the package.

2.4.3 Building the Package

After creating the package, navigate back to the root of your workspace and build the package using `colcon`:

```
$ cd ~/ros2_ws
$ colcon build --packages-select s2_py_pubsub
```

Alternatively, to build all packages in the workspace:

```
$ colcon build
```

2.4.4 Examining the Package Contents

Navigate to the newly created package directory, `s2_py_pubsub`, and list its contents:

```
$ cd src/s2_py_pubsub
$ ls
```

You should see the following key files and directories:

- `s2_py_pubsub/`: Directory with the same name as your package, used by ROS 2 tools to find your package, contains `__init__.py`.
- `package.xml`: Defines metadata about the package, including its name, version, description, licenses, maintainers, authors, and dependencies.
- `setup.py`: Script for installing and setting up the package using Python's `setuptools`.
- `setup.cfg`: Configuration file for `setuptools`.
- `resource/`: Directory containing resource files.
- `test/`: Directory designated for test files.

2.4.5 Detailed Examination of Key Package Files

`package.xml`

The `package.xml` file contains essential metadata about the ROS 2 package. Key elements include:

- `<name>`: Specifies the package's name.
- `<version>`: Indicates the current version of the package.
- `<description>`: Provides a brief overview of the package's purpose.
- `<maintainer>`: Contains contact information for the package maintainer.
- `<license>`: Details the licensing information.
- `<depend>`: Lists build-time and runtime dependencies required by the package.

To examine the `package.xml` file, use the following command:

```
$ code package.xml
```

Ensure that the dependencies listed here match those specified during package creation.

`setup.py`

The `setup.py` script utilizes `setuptools` to configure how the package is installed and structured.. It specifies details such as the package name, version, author information, license, and included packages or modules.

To inspect the `setup.py` file, execute:

```
$ code setup.py
```

2.4.6 Ensuring Consistency Between `package.xml` and `setup.py`

It is crucial that the information in `package.xml` and `setup.py` is consistent. Inconsistencies may cause build failures or unexpected runtime behavior. Ensure that details like the package name, version, description, maintainer, license, and dependencies align across both files.

By following these steps, you have created a Python-based ROS 2 package, built it, and examined its structure and configuration files. This foundational setup is essential for developing and organizing your ROS 2 Python nodes effectively.

For more details on package creation and management, refer to the official Creating a ROS 2 Package tutorial.

2.5 Creating a ROS 2 Publisher Node with C++

2.5.1 Setting Up the C++ Node

Navigating to the Package Directory

Open a terminal and navigate to your C++ package directory within your ROS 2 workspace (e.g., `~/ros2_ws`):

```
$ cd ros2_ws/src/s2_cpp_pubsub/src
```

Replace `s2_cpp_pubsub` with your actual package name.

Creating the C++ Node File

Create a new C++ file for your publisher node, for example, `publisher.cpp`:

```
$ touch publisher.cpp
```

Examine the folder contents to verify the new file has been created:

```
$ ls
```

You should now see the `publisher.cpp` file.

2.5.2 Editing the C++ Node

Opening the Workspace in VS Code

Navigate back to the root of your workspace:

```
$ cd ../../..
```

Then, open the workspace in Visual Studio Code:

```
$ code .
```

In VS Code, locate and open the `publisher.cpp` file for editing.

Implementing the Node Code

Below is an example implementation of a simple ROS 2 publisher node in C++:

```cpp
#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

/* This example creates a subclass of Node and uses std::bind() to register a
* member function as a callback from the timer. */

class CppPublisher : public rclcpp::Node {
    public:
        CppPublisher():
        Node("cpp_publisher_node"), count_(0) {
            RCLCPP_INFO(this->get_logger(), "C++ Publisher node has been started");
            publisher_ = this->create_publisher<std_msgs::msg::String>("cpp_str_topic",
↪  10);
            timer_ = this->create_wall_timer(
            500ms, std::bind(&CppPublisher::timer_callback, this));
        }
```

```
    private:
        void timer_callback() {
            std_msgs::msg::String message = std_msgs::msg::String();
            message.data = "Hello, this is Eduardo from the C++ Publisher: " +
↪   std::to_string(count_++);
            RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
            publisher_->publish(message);
        }
        rclcpp::TimerBase::SharedPtr timer_;
        rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
        size_t count_;
};

int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    std::shared_ptr<rclcpp::Node> node = std::make_shared<CppPublisher>();
    rclcpp::spin(node);
    node.reset();
    rclcpp::shutdown();
    return 0;
}
```

This code defines a node that publishes a "Hello, World" message along with a counter to the topic `cpp_str_topic` every 0.5 seconds.

Breaking Down the Publisher Node Code

We now examine each part of the code in detail to understand how the ROS 2 C++ publisher node works.

1. Includes and Namespace

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;
```

- `<chrono>` and `<memory>` are used for time management and smart pointers.
- `<functional>` and `<string>` support function binding and string operations.
- `rclcpp/rclcpp.hpp` provides the ROS 2 C++ API.
- `std_msgs/msg/string.hpp` defines the standard String message type.
- The `usingnamespacestd::chrono_literals;` statement allows the use of time literals (e.g., `500ms`) for time durations.

2. Defining the Node Class

```
class CppPublisher : public rclcpp::Node {
    public:
        CppPublisher():
        Node("cpp_publisher_node"), count_(0) {
            RCLCPP_INFO(this->get_logger(), "C++ Publisher node has been started");
            publisher_ = this->create_publisher<std_msgs::msg::String>("cpp_str_topic",
↪  10);
            timer_ = this->create_wall_timer(
            500ms, std::bind(&CppPublisher::timer_callback, this));
        }
```

- The `CppPublisher` class inherits from `rclcpp::Node` to create a new ROS 2 node.
- `public:` This access specifier indicates that all members declared after it (until another access specifier is encountered) are publicly accessible. This means the constructor and any public methods can be accessed from outside the class.
- The constructor `CppPublisher()` initializes the node with the name `"cpp_publisher_node"` and sets the counter `count_` to 0.
- `RCLCPP_INFO(this->get_logger(),"C++Publishernodehasbeenstarted");` logs an informational message.
- A publisher is created to send `std_msgs::msg::String` messages on the `"cpp_str_topic"` topic with a queue size of 10.
- A wall timer is set up to call the `timer_callback` method every 500 milliseconds.

3. Timer Callback Method

```
    private:
        void timer_callback() {
            std_msgs::msg::String message = std_msgs::msg::String();
            message.data = "Hello, this is Eduardo from the C++ Publisher: " +
↪  std::to_string(count_++);
            RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
            publisher_->publish(message);
        }
        rclcpp::TimerBase::SharedPtr timer_;
        rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
        size_t count_;
};
```

- `private:` This access specifier indicates that all members declared after it are accessible only within the class itself. This encapsulation prevents external access to these members, ensuring they can only be modified or called by member functions of the class.
- The `timer_callback` method is invoked periodically by the timer. It creates a new message, sets its `data` field to include the current counter value, logs the message, and publishes it on the topic `cpp_str_topic`.
- `rclcpp::TimerBase::SharedPtrtimer_;` is a shared pointer to a timer object that triggers the callback at regular intervals.
- `rclcpp::Publisher<std_msgs::msg::String>::SharedPtrpublisher_;` is a shared pointer to a publisher that sends messages of type `std_msgs::msg::String`.

- `size_tcount_;` is a counter used to generate a unique message each time the callback is invoked.

4. The Main Function

```
int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    std::shared_ptr<rclcpp::Node> node = std::make_shared<CppPublisher>();
    rclcpp::spin(node);
    node.reset();
    rclcpp::shutdown();
    return 0;
}
```

- `rclcpp::init(argc,argv)` initializes ROS 2 communication by parsing command-line arguments and setting up necessary middleware.
- `node=std::make_shared<CppPublisher>()` creates an instance of the `CppPublisher` node.
- `rclcpp::spin(node)` keeps the node active, processing callbacks (such as `timer_callback()`) until a shutdown signal is received.
- `node.reset()` releases the shared pointer to the node so that its resources can be deallocated and the destructor can be called before ROS 2 shuts down.
- `rclcpp::shutdown()` gracefully shuts down ROS 2, releasing all allocated resources when the node stops.
- `return0;` indicates successful program termination.

This detailed breakdown explains each component of the ROS 2 C++ publisher node. For further details and additional context, refer to the Creating ROS 2 Nodes (C++) tutorial.

2.5.3 Integrating the Publisher Node into the Package

Modifying `CMakeLists.txt`

To enable ROS 2 to build and run your C++ node, modify the `CMakeLists.txt` file of your package. Open `CMakeLists.txt` in VS Code and add or verify the following lines:

```
# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(publisher_exe src/publisher.cpp)
ament_target_dependencies(publisher_exe rclcpp std_msgs)

install(TARGETS
    publisher_exe
    DESTINATION lib/${PROJECT_NAME}
)
```

Breaking down each line:

- `find_package(ament_cmakeREQUIRED)` locates the ament build system.

- `find_package(rclcppREQUIRED)` and `find_package(std_msgsREQUIRED)` ensure that the required dependencies are found.
- `add_executable(publisher_exesrc/publisher.cpp)` creates an executable from your source file.
- `ament_target_dependencies(publisher_exerclcppstd_msgs)` links the required libraries.
- The `install` command specifies where the executable should be installed.

Building the Package

After saving the changes to `CMakeLists.txt`, build your package. In the current terminal session, from the root of the workspace, compile with:

```
$ colcon build
```

Alternatively, to build only the C++ package:

```
$ colcon build --packages-select s2_cpp_pubsub
```

If the build completes successfully, you will see the following confirmation message: `'colcon build' successful`.

2.5.4 Running the Publisher Node as a Standalone Executable (Optional)

You can run your C++ publisher node outside of the ROS 2 environment. Open a new terminal (or use an additional session in Terminator) and navigate to the location where the executable was installed. This could be either:

```
$ cd build/s2_cpp_pubsub
```

or:

```
$ cd install/s2_cpp_pubsub/lib/s2_cpp_pubsub
```

as the `install` command specified, in the `CMakeLists.txt` file, where the executable should be installed. Then, run the executable:

```
$ ./publisher_exe
```

as the `add_executable` command specified, in the `CMakeLists.txt` file, how the executable should be named.

Sourcing the Environment

Before running the node, in the first (or current) terminal session from the root of the workspace, source the workspace's setup file to load the necessary environment variables:

```
$ source ./install/setup.bash
```

This step ensures that ROS 2 and the built packages are properly configured.

2.5.5 Running the ROS 2 Publisher Node with ROS 2

Execute your C++ publisher node using the `ros2run` command:

```
$ ros2 run s2_cpp_pubsub publisher_exe
```

At this point, if everything is set up correctly, your node should be actively publishing messages to the `cpp_str_topic` topic.

2.5.6 Summary of Key Identifiers

At the end of the creation and configuration of the ROS 2 C++ publisher node, it is important to distinguish between the following identifiers used:

1. `publisher.cpp`: The filename of your C++ source file.
2. `cpp_publisher_node`: The name of the node as defined in the constructor of your C++ class.
3. `publisher_exe`: The name of the executable specified in `CMakeLists.txt` under the `add_executable` command.

Ensuring these identifiers are correctly set and consistently used is crucial for the proper functioning of your ROS 2 C++ nodes.

For more details on node creation and setup in C++, refer to the official Creating ROS 2 Nodes (C++) tutorial.

2.6 Creating a ROS 2 Subscriber Node with C++

2.6.1 Setting Up the C++ Node

Navigating to the Package Directory

Open a terminal and navigate to your C++ package directory within your ROS 2 workspace (e.g., `~/ros2_ws`):

```
$ cd ros2_ws/src/s2_cpp_pubsub/src
```

Replace `s2_cpp_pubsub` with your actual package name.

Creating the C++ Node File

Create a new C++ file for your subscriber node, for example, `subscriber.cpp`:

```
$ touch subscriber.cpp
```

Then, list the folder contents to verify the file has been created:

```
$ ls
```

You should now see the `subscriber.cpp` file.

2.6.2 Editing the C++ Node

Opening the Workspace in VS Code

Navigate back to the root of your workspace:

```
$ cd ../../..
```

Then, open the workspace in Visual Studio Code:

```
$ code .
```

In VS Code, locate and open the `subscriber.cpp` file for editing.

Implementing the Node Code

Below is an example implementation of a simple ROS 2 subscriber node in C++:

```cpp
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using std::placeholders::_1;

class CppSubscriber : public rclcpp::Node
{
    public:
        CppSubscriber()
        : Node("cpp_subscriber_node") {
            RCLCPP_INFO(this->get_logger(), "C++ Subscriber node has been started");
            subscription_ = this->create_subscription<std_msgs::msg::String>(
            "py_str_topic", 10, std::bind(&CppSubscriber::topic_callback, this, _1));
        }

    private:
        void topic_callback(const std_msgs::msg::String & msg) {
            RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
        }
        rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    std::shared_ptr<rclcpp::Node> node = std::make_shared<CppSubscriber>();
    rclcpp::spin(node);
    node.reset();
    rclcpp::shutdown();
    return 0;
}
```

This code defines a node that subscribes to the topic `py_str_topic` and logs the messages received.

Breaking Down the Subscriber Node Code

1. Includes and Namespace

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using std::placeholders::_1;
```

- `<memory>` is used for smart pointers.
- `rclcpp/rclcpp.hpp` provides the ROS 2 C++ API.
- `std_msgs/msg/string.hpp` defines the standard String message type.
- `usingstd::placeholders::_1;` simplifies binding the callback function.

2. Defining the Node Class

```
class CppSubscriber : public rclcpp::Node
{
    public:
        CppSubscriber()
        : Node("cpp_subscriber_node") {
            RCLCPP_INFO(this->get_logger(), "C++ Subscriber node has been started");
            subscription_ = this->create_subscription<std_msgs::msg::String>(
            "py_str_topic", 10, std::bind(&CppSubscriber::topic_callback, this, _1));
        }
```

- The `CppSubscriber` class inherits from `rclcpp::Node` to create a new ROS 2 node.
- `public:` makes the constructor accessible outside the class.
- The constructor `CppSubscriber()` initializes the node with the name `"cpp_subscriber_node"` and creates a subscription to the `"py_str_topic"` topic with a queue size of 10.
- The subscription's callback is bound using `std::bind`.

3. Listener Callback Method

```
    private:
        void topic_callback(const std_msgs::msg::String & msg) {
            RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg.data.c_str());
        }
        rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};
```

- `private:` restricts access to the callback and the subscription pointer, encapsulating internal details.
- The `topic_callback` method is invoked when a new message is received. It logs the content of the message.

4. The Main Function

```
int main(int argc, char * argv[]) {
    rclcpp::init(argc, argv);
    std::shared_ptr<rclcpp::Node> node = std::make_shared<CppSubscriber>();
    rclcpp::spin(node);
    node.reset();
    rclcpp::shutdown();
    return 0;
}
```

- `rclcpp::init(argc,argv)` initializes ROS 2 communication by parsing command-line arguments and setting up necessary middleware.
- `node=std::make_shared<CppSubscriber>()` creates an instance of the `CppSubscriber` node.
- `rclcpp::spin(node)` keeps the node active, processing callbacks (such as `topic_callback()`) until a shutdown signal is received.
- `node.reset()` releases the shared pointer to the node so that its resources can be deallocated and the destructor can be called before ROS 2 shuts down.
- `rclcpp::shutdown()` gracefully shuts down ROS 2, releasing all allocated resources when the node stops.
- `return0;` indicates successful program termination.

This detailed breakdown explains each component of the ROS 2 C++ subscriber node. For further details and additional context, refer to the Creating ROS 2 Nodes (C++) tutorial.

2.6.3 Integrating the Subscriber Node into the Package

Modifying `CMakeLists.txt`

To build and run your C++ subscriber node, modify the `CMakeLists.txt` file of your package. Open `CMakeLists.txt` in VS Code and add or verify the following lines, keeping any other existing ROS 2 node setup (e.g., `publisher_exe`):

```
# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(publisher_exe src/publisher.cpp)
ament_target_dependencies(publisher_exe rclcpp std_msgs)

add_executable(subscriber_exe src/subscriber.cpp)
ament_target_dependencies(subscriber_exe rclcpp std_msgs)

install(TARGETS
    publisher_exe
    subscriber_exe
    DESTINATION lib/${PROJECT_NAME}
)
```

Breaking down each line:

- `find_package(ament_cmakeREQUIRED)` locates the ament build system.
- `find_package(rclcppREQUIRED)` and `find_package(std_msgsREQUIRED)` ensure that the required dependencies are found.
- `add_executable(subscriber_exesrc/subscriber.cpp)` creates an executable from your source file.
- `ament_target_dependencies(subscriber_exerclcppstd_msgs)` links the required libraries.
- The `install` command specifies where the executable should be installed.

Building the Package

After updating `CMakeLists.txt`, build your package. In the current terminal session from the root of your workspace, compile with:

```
$ colcon build
```

Alternatively, to build only the C++ package:

```
$ colcon build --packages-select s2_cpp_pubsub
```

If the build completes successfully, you will see the following confirmation message: `'colcon build' successful`.

2.6.4 Running the Subscriber Node as a Standalone Executable (Optional)

You can run your C++ subscriber node outside of the ROS 2 environment. Open a new terminal (or use an additional session in Terminator) and navigate to the directory where the executable was installed. This could be either:

```
$ cd build/s2_cpp_pubsub
```

or:

```
$ cd install/s2_cpp_pubsub/lib/s2_cpp_pubsub
```

as the `install` command specified, in the `CMakeLists.txt` file, where the executable should be installed. Then, run the executable:

```
$ ./subscriber_exe
```

as the `add_executable` command specified, in the `CMakeLists.txt` file, how the executable should be named.

Sourcing the Environment

Before running the node with ROS 2, in the first (or current) terminal session from the root of your workspace, source the workspace's setup file to load the necessary environment variables:

```
$ source ./install/setup.bash
```

This ensures that ROS 2 and your built packages are properly configured.

2.6.5 Running the ROS 2 Subscriber Node with ROS 2

Execute your C++ subscriber node using the `ros2run` command:

```
$ ros2 run s2_cpp_pubsub subscriber_exe
```

At this point, if everything is set up correctly and your Python publisher node is running, your subscriber node should be actively receiving and logging messages published from the `py_str_topic` topic.

2.6.6 Summary of Key Identifiers

At the end of the creation and configuration of the ROS 2 C++ subscriber node, it is important to distinguish between the following identifiers used:

1. `subscriber.cpp`: The filename of your C++ source file.
2. `cpp_subscriber_node`: The name of the node as defined in the constructor of your C++ class.
3. `subscriber_exe`: The name of the executable specified in `CMakeLists.txt` under the `add_executable` command.

Ensuring these identifiers are correctly set and consistently used is crucial for the proper functioning of your ROS 2 C++ nodes.

For more details on node creation and setup in C++, refer to the official Creating ROS 2 Nodes (C++) tutorial.

2.7 Creating a ROS 2 Publisher Node with Python

2.7.1 Setting Up the Python Node

Navigating to the Package Directory

Open a terminal and navigate to your Python package directory within the ROS 2 workspace (e.g., `~/ros2_ws`):

```
$ cd ros2_ws/src/s2_py_pubsub/s2_py_pubsub
```

Replace `s2_py_pubsub` with your actual package name.

Creating the Python Node File

Create a new Python file for your publisher node, for example, `publisher.py`:

```
$ touch publisher.py
```

Examine the folder contents to verify the new file has been created:

```
$ ls
```

You should now see both the `__init__.py` and `publisher.py` files.

Making the Python File Executable

Ensure the new Python file is executable:

```
$ chmod +x publisher.py
```

Examine the folder contents again to confirm that `publisher.py` now appears in a different color, indicating that it is executable.

2.7.2 Editing the Python Node

Opening the Workspace in VS Code

Navigate back to the root of your workspace:

```
$ cd ../../..
```

Then, open the workspace in Visual Studio Code:

```
$ code .
```

In VS Code, locate and open the `publisher.py` file for editing.

Implementing the Node Code

Below is an example implementation of a simple ROS 2 publisher node in Python:

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class PyPublisher(Node):

    def __init__(self):
        super().__init__('py_publisher_node')
        self.get_logger().info("Python Publisher node has been started")
        self.publisher_ = self.create_publisher(String, 'py_str_topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def timer_callback(self):
        msg = String()
        msg.data = 'Hello, this is Eduardo from the Python Publisher: %d' % self.i
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.publisher_.publish(msg)
        self.i += 1
```

```
# To ensure that resources are properly cleaned up and that
# all intended statements are executed upon shutdown,
# you should handle the KeyboardInterrupt exception.
def main(args=None):
    rclpy.init(args=args)
    node = PyPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

This script defines a node that publishes a "Hello, World" message along with a counter to the topic named `py_str_topic` every 0.5 seconds.

Breaking Down the Publisher Node Code

We now examine each part of the code in detail to understand how the ROS 2 Python publisher node works.

1. Shebang and Imports

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
```

- The shebang line (`#!/usr/bin/env python3`) ensures that the script is executed with Python 3.
- `rclpy` is the ROS 2 Python client library, which provides the tools needed to write ROS 2 nodes.
- `Node` is the base class for creating ROS 2 nodes.
- `String` is the message type imported from the standard messages package.

2. Defining the Node Class

```
class PyPublisher(Node):

    def __init__(self):
        super().__init__('py_publisher_node')
        self.get_logger().info("Python Publisher node has been started")
        self.publisher_ = self.create_publisher(String, 'py_str_topic', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0
```

- The `PyPublisher` class inherits from `Node` to create a new ROS 2 node.
- In the `__init__` method:

- `super().__init__()` initializes the node with the name `py_publisher_node`.
- `self.get_logger().info("PythonPublishernodehasbeenstarted")` logs an informational message when the node starts.
- `self.create_publisher(String,'py_str_topic',10)` creates a publisher that will publish messages of type `String` to the topic `py_str_topic` with a queue size of 10.
- `self.create_timer(timer_period,self.timer_callback)` sets up a timer to call the `timer_callback` method every 0.5 seconds.
- `self.i=0` initializes a counter for use in the published messages.

3. Timer Callback Method

```
    def timer_callback(self):
        msg = String()
        msg.data = 'Hello, this is Eduardo from the Python Publisher: %d' % self.i
        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg.data)
        self.i += 1
```

- The `timer_callback` method is called periodically by the timer.
- A new `String` message is created, and its `data` field is set to include the current counter value.
- The message is published to `py_str_topic`.
- An informational log displays the published message.
- The counter `self.i` is incremented for the next callback.

4. The Main Function

```
# To ensure that resources are properly cleaned up and that
# all intended statements are executed upon shutdown,
# you should handle the KeyboardInterrupt exception.
def main(args=None):
    rclpy.init(args=args)
    node = PyPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

- `rclpy.init(args=args)` initializes ROS 2 communication by parsing command-line arguments and setting up necessary middleware.
- An instance of the `PyPublisher` node is created.
- `rclpy.spin(node)` keeps the node active, processing callbacks until a shutdown signal is received.
- The `try-except-finally` block ensures graceful shutdown:

  - `try`: Runs the spinning loop to keep the node active.
  - `exceptKeyboardInterrupt`: Catches the interrupt signal (e.g., `Ctrl+C`) and explicitly destroys the node to release resources.
  - `finally`: Ensures that `rclpy.shutdown()` is called to clean up resources, regardless of how the spin loop is exited.
- `if__name__==’__main__’`: Ensures that the `main()` function is called only when the script is executed directly.

This detailed breakdown explains each component of the ROS 2 Python publisher node. For further details and additional context, refer to the Creating ROS 2 Nodes (Python) tutorial.

2.7.3 Running the Publisher Node Standalone (Optional)

After editing the file, you can run your Python publisher node out of ROS 2 environment. For this, open a new terminal (or use an additional session in Terminator) and navigate to the location of the Python file:

```
$ cd src/s2_py_pubsub/s2_py_pubsub
```

Then, test the Python file by running it with one of the following commands:

```
$ ./publisher.py
```

to run it as an executable, or

```
$ python3 publisher.py
```

to run it as a Python script.

2.7.4 Integrating the Publisher Node into the Package

Modifying `setup.py`

To enable ROS 2 to recognize and execute your publisher node, you need to specify it in the `setup.py` file of your package. Open `setup.py` in VS Code and locate the `entry_points` field. Then, add your node as follows:

```
entry_points={
    'console_scripts': [
        'publisher_exe = s2_py_pubsub.publisher:main',
    ],
},
```

Breaking down the components of this specification:

- `publisher_exe`: Specifies the command-line executable name that you will use to run the node.
- `s2_py_pubsub`: Indicates the name of your package.
- `publisher`: Specifies the Python file (without the `.py` extension) containing the node.
- `main`: Refers to the function that will be executed when the node runs.

Building the Package

After saving the changes to `setup.py`, build your package. In the first terminal session, from the root of the workspace, compile with:

```
$ colcon build
```

Alternatively, to build only the Python package:

```
$ colcon build --packages-select s2_py_pubsub
```

For development purposes, consider using the `--symlink-install` option so that changes in the source files are immediately reflected without requiring a full rebuild:

```
$ colcon build --symlink-install
```

or, more specifically:

```
$ colcon build --packages-select s2_py_pubsub --symlink-install
```

If the build completes successfully, you will see the following confirmation message: `'colcon build' successful`.

Sourcing the Environment

Before running the node, in the first (or current) terminal session from the root of the workspace, source the workspace's setup file to load the necessary environment variables:

```
$ source ./install/setup.bash
```

This step ensures that ROS 2 and the built packages are properly configured.

2.7.5 Running the ROS 2 Publisher Node with ROS 2

Execute your Python publisher node using the `ros2run` command:

```
$ ros2 run s2_py_pubsub publisher_exe
```

At this point, if everything is set up correctly, your node should be actively publishing messages to the `py_str_topic` topic.

2.7.6 Summary of Key Identifiers

At the end of the creation and configuration of the ROS 2 publisher node, it is important to distinguish between the different identifiers used:

1. `publisher.py`: The filename of your Python script.
2. `py_publisher_node`: The name of the node as defined in the `super().__init__()` call within your script.
3. `publisher_exe`: The command-line executable name specified in `setup.py` under the `entry_points` field.

Ensuring these identifiers are correctly set and consistently used is crucial for the proper functioning of your ROS 2 Python nodes.

For more details on node creation and setup in Python, refer to the official Creating ROS 2 Nodes (Python) tutorial.

2.8 Creating a ROS 2 Subscriber Node with Python

2.8.1 Setting Up the Python Node

Navigating to the Package Directory

Open a terminal and navigate to your Python package directory within the ROS 2 workspace (e.g., `~/ros2_ws`):

```
$ cd ros2_ws/src/s2_py_pubsub/s2_py_pubsub
```

Replace `s2_py_pubsub` with your actual package name.

Creating the Python Node File

Create a new Python file for your subscriber node, for example, `subscriber.py`:

```
$ touch subscriber.py
```

Examine the folder contents to verify the new file has been created:

```
$ ls
```

You should now see both the `__init__.py` and `subscriber.py` files.

Making the Python File Executable

Ensure the new Python file is executable:

```
$ chmod +x subscriber.py
```

Check the folder contents again to confirm that `subscriber.py` now appears in a different color, indicating that it is executable.

2.8.2 Editing the Python Node

Opening the Workspace in VS Code

Navigate back to the root of your workspace:

```
$ cd ../../..
```

Then, open the workspace in Visual Studio Code:

```
$ code .
```

In VS Code, locate and open the `subscriber.py` file for editing.

Implementing the Node Code

Below is an example implementation of a simple ROS 2 subscriber node in Python:

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class PySubscriber(Node):

    def __init__(self):
        super().__init__('py_subscriber_node')
        self.get_logger().info("Python Subscriber node has been started")
        self.subscription = self.create_subscription(
            String,
            'cpp_str_topic',
            self.listener_callback,
            10)
        self.subscription  # prevent unused variable warning

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)

# To ensure that resources are properly cleaned up and that
# all intended statements are executed upon shutdown,
# you should handle the KeyboardInterrupt exception.
def main(args=None):
    rclpy.init(args=args)
    node = PySubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

This script defines a node that subscribes to the topic `cpp_str_topic` and logs the messages received.

2.8.3 Breaking Down the Subscriber Node Code

1. Shebang and Imports

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
```

- The shebang ensures the script runs with Python 3.
- `rclpy` is the ROS 2 Python client library.
- `Node` is the base class for creating ROS 2 nodes.
- `String` is the message type imported from the standard messages package.

2. Defining the Node Class

```
class PySubscriber(Node):

    def __init__(self):
        super().__init__('py_subscriber_node')
        self.get_logger().info("Python Subscriber node has been started")
        self.subscription = self.create_subscription(
            String,
            'cpp_str_topic',
            self.listener_callback,
            10)
        self.subscription  # prevent unused variable warning
```

- The `PySubscriber` class inherits from `Node` to create a new ROS 2 node.
- The node is initialized with the name `py_subscriber_node`.
- `self.get_logger().info("PythonSubscribernodehasbeenstarted")` logs an informational message when the node starts.
- `self.create_subscription` creates a subscription to the topic `cpp_str_topic` for messages of type `String` with a queue size of 10.
- `self.listener_callback` is set as the callback function to be invoked upon receiving a message.

3. Listener Callback Method

```
    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)
```

- `listener_callback` is executed every time a message is received.
- An informational log displays the data of the received message.

4. The Main Function

```
# To ensure that resources are properly cleaned up and that
# all intended statements are executed upon shutdown,
# you should handle the KeyboardInterrupt exception.
def main(args=None):
    rclpy.init(args=args)
    node = PySubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

- `rclpy.init(args=args)` initializes ROS 2 communication by parsing command-line arguments and setting up necessary middleware.

- An instance of the `PySubscriber` node is created.
- `rclpy.spin(node)` keeps the node active, processing callbacks until a shutdown signal is received.
- The `try-except-finally` block ensures graceful shutdown:
  - `try`: Runs the spinning loop to keep the node active.
  - `exceptKeyboardInterrupt`: Catches the interrupt signal (e.g., `Ctrl+C`) and explicitly destroys the node to release resources.
  - `finally`: Ensures that `rclpy.shutdown()` is called to clean up resources, regardless of how the spin loop is exited.
- `if__name__==’__main__’`: Ensures that the `main()` function is called only when the script is executed directly.

This detailed breakdown explains each component of the ROS 2 Python subscriber node. For further details and additional context, refer to the Creating ROS 2 Nodes (Python) tutorial.

2.8.4 Running the Subscriber Node Standalone (Optional)

After editing the file, you can run your Python subscriber node out of ROS 2 environment. For this, open a new terminal session (or use an additional session in Terminator) and navigate to the location of the Python file:

```
$ cd src/s2_py_pubsub/s2_py_pubsub
```

Then, test the Python file by running it with one of the following commands:

```
$ ./subscriber.py
```

to run it as an executable, or

```
$ python3 subscriber.py
```

to run it as a Python script.

2.8.5 Integrating the Subscriber Node into the Package

Modifying `setup.py`

To allow ROS 2 to recognize and execute your subscriber node, specify it in the `setup.py` file of your package. Open `setup.py` in VS Code and locate the `entry_points` field. Then, add your node entry immediately after any existing ROS 2 node entries (e.g., `publisher_exe`):

```
entry_points={
    'console_scripts': [
        'publisher_exe = s2_py_pubsub.publisher:main',
        'subscriber_exe = s2_py_pubsub.subscriber:main',
    ],
},
```

Breaking down the components of this specification:

- `subscriber_exe`: Specifies the command-line executable name that you will use to run the subscriber node.

- `s2_py_pubsub`: Indicates the name of your package.
- `subscriber`: Specifies the Python file (without the `.py` extension) that contains the node.
- `main`: Refers to the function that will be executed when the node is run.

Building the Package

After updating `setup.py`, build your package. In the first terminal session, from the root of the workspace, compile with:

```
$ colcon build
```

Alternatively, to build only the Python package:

```
$ colcon build --packages-select s2_py_pubsub
```

For development purposes, you might consider using the `--symlink-install` option so that changes in the source files are immediately reflected without requiring a full rebuild:

```
$ colcon build --symlink-install
```

or, more specifically:

```
$ colcon build --packages-select s2_py_pubsub --symlink-install
```

If the build completes successfully, you will see the following confirmation message: `'colcon build' successful`.

Sourcing the Environment

Before running the node, in the first (or current) terminal session from the root of the workspace, source the workspace's setup file to load the necessary environment variables:

```
$ source ./install/setup.bash
```

This ensures that ROS 2 and your built packages are properly configured.

2.8.6 Running the ROS 2 Subscriber Node with ROS 2

Execute your subscriber node using the `ros2run` command:

```
$ ros2 run s2_py_pubsub subscriber_exe
```

At this point, if everything is set up correctly and your C++ publisher node is running, your subscriber node should be actively receiving and logging messages published from the `cpp_str_topic` topic.

2.8.7 Summary of Key Identifiers

It is important to distinguish between the following identifiers in your ROS 2 subscriber node:

1. `subscriber.py`: The filename of your Python script.
2. `py_subscriber_node`: The name of the node as defined in the `super().__init__()` call within your script.
3. `subscriber_exe`: The command-line executable name specified in `setup.py` under the `entry_points` field.

Ensuring that these identifiers are correctly set and consistently used is crucial for the proper functioning of your ROS 2 Python nodes.

For more details on creating ROS 2 nodes in Python, refer to the official Creating ROS 2 Nodes (Python) tutorial.

2.9 Practice Assignment: Running ROS 2 Nodes

In this assignment, you will run ROS 2 publisher and subscriber nodes implemented in both C++ and Python, as described in Sections 2.5, 2.6, 2.7, and 2.8. You will then capture evidence of successful execution using a screenshot and submit it in the designated assignment area on Canvas (within the ROS 2 module) as a PNG file. The filename must follow this format:

`FirstnameLastname_evidence_sX.png`,

where `sX` indicates the session number. For example, in this case, the correct filename would be:

`EduardoDavila_evidence_s2.png`.

2.9.1 Executing ROS 2 Publishers and Subscribers

Running the Python Publisher and Subscriber

After implementing and saving your Python nodes and updating `package.xml` and `setup.py` files, build your package and run the nodes. In two separate terminal sessions (or using the `Terminator` emulator), execute the following commands from the root of your workspace:

- In **Terminal 1**, build your package, source the setup file, and run the Python publisher node:

```
$ colcon build --packages-select s2_py_pubsub
$ source install/setup.bash
$ ros2 run s2_py_pubsub publisher_exe
```

- In **Terminal 2**, source the setup file and run the Python subscriber node:

```
$ source install/setup.bash
$ ros2 run s2_py_pubsub subscriber_exe
```

You should observe that the `py_publisher` node sends a personalized message (e.g., displaying your name), and the `py_subscriber` node awaits the reception of the messages.

Running the C++ Publisher and Subscriber

After implementing and saving your C++ nodes and updating `package.xml` and `CMakeLists.txt` files, build your package and run the nodes. In two additional terminal sessions (or another two sessions in `Terminator`), execute the following commands from the root of your workspace:

- In **Terminal 3**, build your package, source the setup file, and run the C++ publisher node:

```
$ colcon build --packages-select s2_cpp_pubsub
$ source install/setup.bash
$ ros2 run s2_cpp_pubsub publisher_exe
```

- In **Terminal 4**, source the setup file and run the C++ subscriber node:

```
$ source install/setup.bash
$ ros2 run s2_cpp_pubsub subscriber_exe
```

Here, you should observe that the `py_publisher` node publishes your personalized message (e.g., displaying your name), whereas the `cpp_subscriber` node confirms its reception. Additionally, the `cpp_publisher` node publishes your other personalized message (e.g., displaying your name), whereas the `py_subscriber` node confirms its reception.

Visualizing the ROS Graph with `rqt_graph`

To ensure that your publisher and subscriber nodes are correctly connected, use the `rqt_graph` tool. Then, in **Terminal 5**, execute the following command (without the need to enter the workspace directory):

```
$ ros2 run rqt_graph rqt_graph
```

or simply:

```
$ rqt_graph
```

This visualization should show the topics connecting your publisher and subscriber nodes as described above.

2.9.2 Submission Instructions

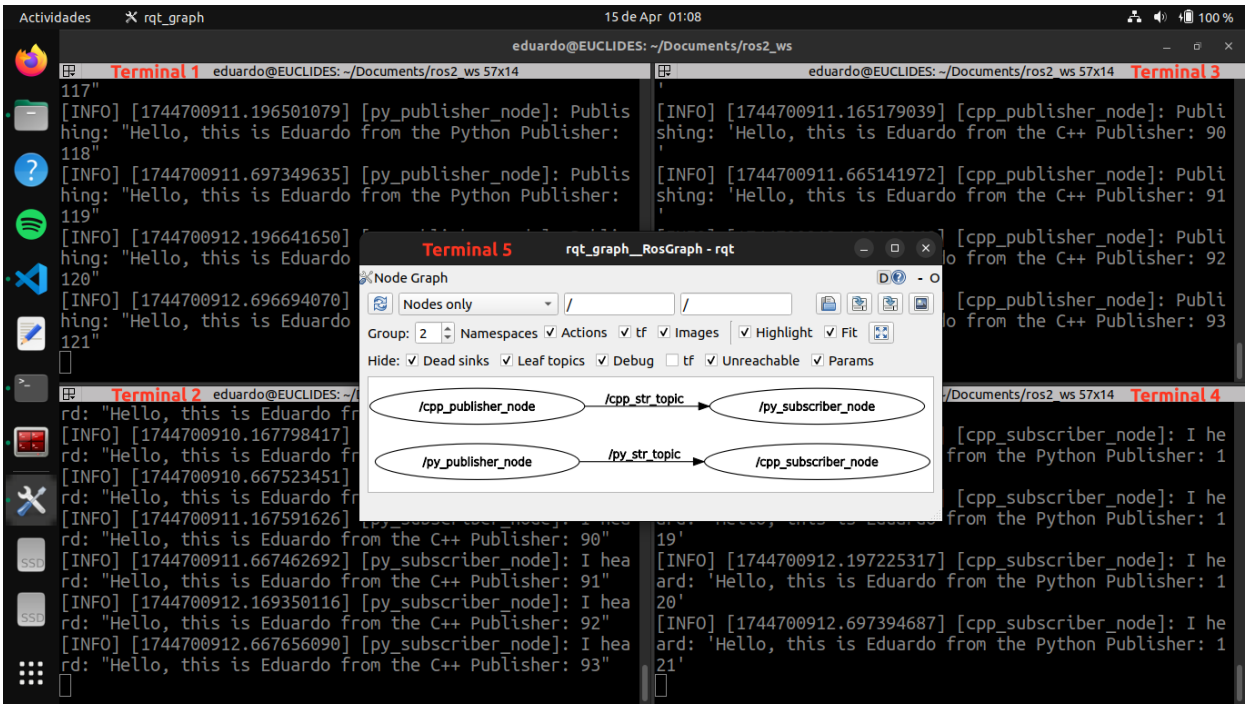

Figure 2.1: Example screenshot showing terminal outputs of the publisher and subscriber nodes, in C++ and Python, in action.

1. Run the ROS 2 nodes as described above.
2. Capture a **screenshot showing the terminal outputs** from all four nodes (Python and C++ publishers and subscribers) along with the visualization of the ROS graph using the `rqt_graph` tool. See Figure 2.1 for reference.
3. Save the screenshot as a PNG file.
4. Name the file following this format: `FirstnameLastname_evidence_sX.png` (replace `X` with the week number and `Y` with the session number).
5. Submit your file as instructed by the course guidelines on Canvas.

Note: Your machine's username and device name must be visible in the screenshot to verify the authenticity of the submission, as shown in Figure 2.1. Any submission that appears copied, unclear, or altered will be considered invalid and may result in a score of 0 for this assignment.

CHAPTER 3

ROS 2 Service and Client

APPENDIX A

Essential ROS 2 Command Reference

Author: Dr. Eduardo de Jesús Dávila Meza.
EduardoDavila-AI-PhD

This appendix compiles the commands utilized throughout the course for creating and configuring workspaces, packages, nodes, and using integrated tools in ROS 2.

A.1 System Package Management and Updates in Ubuntu

Command	Description
`sudo apt update`	Updates the list of available packages and their versions.
`sudo apt upgrade`	Installs the latest versions of all installed packages.
`sudo apt install <pkg_name>`	Installs the specified package.
`sudo apt install ./<file>.deb`	Installs a package from a local `.deb` file.
`sudo apt install ros-humble-desktop`	Installs the full desktop version of ROS 2 Humble.
`sudo apt install ros-dev-tools`	Installs development tools for ROS 2.
`sudo apt install ros-humble-turtlesim`	Installs the `turtlesim` package for ROS 2 Humble.
`sudo apt install python3-colcon-common-extensions`	Installs common extensions for `colcon` to facilitate package building.
`sudo apt install terminator`	Installs the Terminator terminal emulator.
`sudo snap install tree`	Installs the command-line utility that lists the contents of directories in a recursive, tree-like, and hierarchical format.

Table A.I: Commands for system package management and updates in Ubuntu.

A.2 ROS 2 Environment Setup

Command	Description
`source /opt/ros/humble/setup.bash`	Sets up the environment for ROS 2 Humble in the current terminal session.
`source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash`	Sets up the autocompletion for `colcon` in the current terminal session.
`echo "source/opt/ros/humble/setup.bash" >> ~/.bashrc`	Adds the ROS 2 Humble environment setup to the `.bashrc` file for future terminal sessions.
`echo "source/usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash" >> ~/.bashrc`	Enables autocompletion for `colcon` in the `.bashrc` file for future terminal sessions.
`source ~/.bashrc`	Reloads the `.bashrc` file to apply changes without restarting the terminal.
`cat ~/.bashrc`	Displays the contents of the `.bashrc` file.
`gedit ~/.bashrc`	Opens the `.bashrc` file in the `gedit` text editor.

Table A.II: Commands for ROS 2 environment setup.

A.3 Workspace Creation and Building

Command	Description
`mkdir -p <ws_name>/src`	Creates the workspace directory and its `src` subdirectory.
`cd <ws_name>`	Navigates to the workspace directory.
`colcon build`	Builds all packages in the workspace.
`colcon build --symlink-install`	Builds packages using symbolic links, useful for development purposes.
`source ./install/setup.bash`	Sets up the environment for the built workspace in the current terminal session.
`code .`	Opens the current directory in Visual Studio Code.

Table A.III: Commands for workspace creation and building in ROS 2.

A.4 Package Creation and Editing in C++ and Python

Command	Description
`cd <ws_name>/src`	Navigates to the workspace's `src` directory.
`ros2 pkg create <pkg_name> --build-type ament_cmake --dependencies rclcpp std_msgs --license Apache-2.0 --description "Your package description here"`	Creates a new C++ package with specified dependencies, license, and description.
`ros2 pkg create <pkg_name> --build-type ament_python --dependencies rclpy std_msgs --license Apache-2.0 --description "Your package description here"`	Creates a new Python package with specified dependencies, license, and description.
`ls`	Lists the files and directories in the current directory.
`cd ..`	Navigates one directory up.
`rosdep install -i --from-path src --rosdistro humble -y`	In the root of the workspace, checks for missing dependencies before building.
`colcon build --packages-select <pkg_name>`	Builds only the `<pkg_name>` package.
`colcon build --packages-select <pkg_name> --symlink-install`	Builds the `<pkg_name>` package with symbolic link installation.
`cd src/<pkg_name>`	Navigates to the `<pkg_name>` directory.
`code package.xml`	Opens the `package.xml` file in Visual Studio Code.
`code CMakeLists.txt`	Opens the `CMakeLists.txt` file in Visual Studio Code.
`code setup.py`	Opens the `setup.py` file in Visual Studio Code.
`cd <ws_name>/src/<pkg_name>/src`	Navigates to the `src` subdirectory of a C++ package.
`cd <ws_name>/src/<pkg_name>/<pkg_name>`	Navigates to the `<pkg_name>` subdirectory of a Python package.
`touch <filename>.cpp`	Creates a new C++ source file named `<filename>`.
`touch <filename>.py`	Creates a new Python source file named `<filename>`.
`cd ../../..`	Navigates three directories up.

Table A.IV: Package creation and editing in C++ and Python.

A.5 Dev & Debug Essentials: CLI/GUI Commands

Command	Description
`ros2 run <pkg_name> <node_name>`	Runs the specified node from the specified package.
`ros2 run <pkg_name> <node_name> <args>`	Runs the specified node from the specified package with specified arguments.
`ros2 service call <service_name> <service_type> <args>`	Calls the service `<service_name>` from the specified package with specified arguments.
`ros2 node list`	Lists all active nodes in the ROS 2 system.
`ros2 node info <node_name>`	Displays detailed information about the specified node.
`ros2 topic list`	Lists all active topics in the ROS 2 system.
`ros2 topic info <topic_name>`	Displays detailed information about the specified topic.
`ros2 topic echo <topic_name>`	Echoes the messages of a specified topic.
`ros2 topic pub <topic_name> <msg_type> <args>`	Publishes a message to the specified topic.
`ros2 topic hz topic_name`	Displays the message frequency of a specified topic.
`ros2 interface show <pkg_name>/msg/<custom_msg_name>`	Shows the field structure of a custom message type in the specified package.
`ros2 interface show <pkg_name>/srv/<custom_srv_name>`	Shows the request/response structure of a custom service type in the specified package.
`ros2 run rqt_graph rqt_graph rqt_graph`	Runs the `rqt_graph` tool to visualize the ROS 2 nodes and topics.
`ros2 run rqt_service_caller rqt_service_caller`	Runs the `rqt_service_caller` tool to call services in a GUI.

Table A.V: Essential command-line and graphical interface commands for running nodes, interacting with topics and services, and debugging ROS 2 systems.

APPENDIX B

Using the `geometry_msgs` Package in C++ and Python

Author: Dr. Eduardo de Jesús Dávila Meza.
EduardoDavila-AI-PhD

This appendix provides an overview of how to utilize the `geometry_msgs` package in both C++ and Python. This package defines standardized message types for common geometric primitives such as points, vectors, quaternions, and poses, facilitating spatial data representation and interoperability throughout the ROS 2 ecosystem.

B.1 Importing the Package

B.1.1 In C++

To use the `geometry_msgs` in C++, include the necessary header files in your source code:

```
#include <geometry_msgs/msg/point.hpp>
#include <geometry_msgs/msg/quaternion.hpp>
#include <geometry_msgs/msg/pose.hpp>
```

Each message type corresponds to its own header file within the `geometry_msgs/msg` directory.

B.1.2 In Python

In Python, import the required message classes as follows:

```
from geometry_msgs.msg import Point, Quaternion, Pose
```

B.1.3 Including geometry_msgs in Your Package Dependencies

To use the geometry_msgs message types in your ROS 2 package, you must explicitly declare it as a dependency in both CMakeLists.txt (for C++) and package.xml (for both C++ and Python packages):

In CMakeLists.txt:

```
find_package(geometry_msgs REQUIRED)

ament_target_dependencies(<node_name> geometry_msgs)
```

In package.xml:

```
<depend>geometry_msgs</depend>
```

B.2 Using Message Definitions

The following examples demonstrate how to create, assign, and access values for the most commonly used message types from geometry_msgs.

B.2.1 Point

Represents a 3D position using Cartesian coordinates x, y, and z.

C++

```cpp
#include <geometry_msgs/msg/point.hpp>

geometry_msgs::msg::Point point;
point.x = 1.0;
point.y = 2.0;
point.z = 3.0;

// Accessing values
double x_value = point.x;
double y_value = point.y;
double z_value = point.z;
```

Python

```python
from geometry_msgs.msg import Point

point = Point()
point.x = 1.0
point.y = 2.0
point.z = 3.0

# Accessing values
x_value = point.x
y_value = point.y
z_value = point.z
```

B.2.2 Quaternion

Represents an orientation in free space using quaternion components `x`, `y`, `z`, and `w`. This representation avoids the gimbal lock issues found in Euler angles and supports smooth interpolations.

- `x`, `y`, and `z`: Vector part of the quaternion, defining the axis of rotation.
- `w`: Scalar part, representing the amount of rotation around the axis.
- A unit quaternion (with magnitude 1) is typically required for valid orientation.

C++

```cpp
#include <geometry_msgs/msg/quaternion.hpp>

geometry_msgs::msg::Quaternion quaternion;
quaternion.x = 0.0;
quaternion.y = 0.0;
quaternion.z = 0.0;
quaternion.w = 1.0;

// Accessing values
double w_value = quaternion.w;
```

Python

```python
from geometry_msgs.msg import Quaternion

quaternion = Quaternion()
quaternion.x = 0.0
quaternion.y = 0.0
quaternion.z = 0.0
quaternion.w = 1.0

# Accessing values
w_value = quaternion.w
```

B.2.3 Pose

Combines a `position` (as a Point) and `orientation` (as a Quaternion) to define a complete 6-DOF pose in 3D space.

C++

```cpp
#include <geometry_msgs/msg/pose.hpp>

geometry_msgs::msg::Pose pose;
pose.position.x = 1.0;
pose.position.y = 2.0;
pose.position.z = 3.0;
pose.orientation.x = 0.0;
pose.orientation.y = 0.0;
pose.orientation.z = 0.0;
pose.orientation.w = 1.0;

// Accessing values
double pos_x = pose.position.x;
double orient_w = pose.orientation.w;
```

Python

```python
from geometry_msgs.msg import Pose

pose = Pose()
pose.position.x = 1.0
pose.position.y = 2.0
pose.position.z = 3.0
pose.orientation.x = 0.0
pose.orientation.y = 0.0
pose.orientation.z = 0.0
pose.orientation.w = 1.0

# Accessing values
pos_x = pose.position.x
orient_w = pose.orientation.w
```

B.3 Additional Message Types

The `geometry_msgs` package includes other message types such as `Twist`, `Accel`, and `Wrench`, each with specific fields for representing different geometric concepts. The usage pattern for these messages is similar: import the message type, create an instance, and assign or access the relevant fields accordingly.

For more detailed information on each message type, refer to the ROS 2 official reference: geometry_msgs: Humble.

APPENDIX C

Configuring ROS 2 Package Paths for URDF Visualizer in VS Code

Author: Dr. Eduardo de Jesús Dávila Meza.
EduardoDavila-AI-PhD

To correctly visualize URDF or Xacro files that utilize `package://` URIs (Uniform Resource Identifiers), it is essential to configure the URDF Visualizer extension in Visual Studio Code to recognize the paths of your ROS 2 packages. This ensures that all resources referenced in your robot description files are accurately located and rendered.

C.1 Steps to Add a ROS 2 Package Path

1. Open your project folder in Visual Studio Code.
2. Navigate to the Extensions view by clicking on the Extensions icon in the Activity Bar or pressing `Ctrl+Shift+X`.
3. Search for URDF Visualizer and ensure it is installed.
4. Click on the gear icon (Manage) next to the URDF Visualizer extension and select **Extension Settings**.
5. In the settings, locate the **Urdf-visualizer: Packages** section.
6. Click on **Edit in settings.json** to open the `settings.json` file.
7. Within the `urdf-visualizer.packages` object, add your package path without removing existing entries. You can specify:
   - A relative path:
     ```
     "package_name": "${workspaceFolder}/src/package_name"
     ```
   - An absolute path:
     ```
     "package_name": "/absolute/path/to/package_name"
     ```

8. Save the `settings.json` file and close it.
9. To visualize your URDF or Xacro file:
    - Open the file in VS Code.
    - Click on the eye icon in the top-right corner.
    - Alternatively, press `Ctrl+Shift+P` to open the Command Palette, type `URDF:Preview URDF/Xacro`, and select it.
10. If the URDF Visualizer is already open, click on the reload button to apply the new settings.

C.2 Additional Notes

- If your URDF or Xacro file references multiple packages, ensure all are listed in the `urdf-visualizer.packages` configuration.
- Use absolute paths if your packages are located outside the current workspace.
- The extension supports both URDF and Xacro files, provided the package paths are correctly configured.

For more detailed information, refer to the official documentation: URDF Visualizer Extension.